Knowledge Distillation and Deployment of a Command-Parsing Model on a Local Cloud

Original project by Vivek Sagarwal · Student working copy: Pratham Gupta (Roll no. 230103034)

DRAFT. Items marked PENDING have not been executed yet. Nothing in this report is an invented measurement. Rebuild with make report after replacing every PENDING with a value from results/.

1. Introduction and architecture

A command (e.g. “what is the weather in london”) is parsed into {"intent","date","time","place"}. The teacher (Phi-3.5-mini, Ollama, temperature 0, seed 42) labels the 3,000 SLURP commands in two parallel Kubernetes workers; Pythia-70M is fine-tuned on the cleaned replies (sequence-level distillation, CPU) and served by FastAPI behind a Traefik Ingress on k3d. SeaweedFS (S3 API) holds datasets and model weights; weights are never in the image or Git. Namespaces: data, pipeline, app. **Measured** = taken from files in results/; **assumed** = stated as such.

2. Task 1: Object storage and teacher labelling

Architecture. k3d cluster from cluster/k3d.yaml; SeaweedFS StatefulSet in data; buckets datasets, models; an Indexed Job runs two workers in pipeline (SHARD_INDEX 0 and 1, SHARD_COUNT 2); Ollama runs on the laptop (my choice, because Pods are CPU limited) and is reached via host.k3d.internal. *Implementation.* Sharding: line *n* (from 0) of commands_pool.jsonl goes to worker *n mod SHARD_COUNT*, so the shards are disjoint and together cover every line; each worker writes its own object datasets/teacher/shard-i.jsonl and can resume from it. The prompt (prompts/teacher_prompt.txt) states the schema, the three slot rules and four examples; Ollama’s JSON mode is enabled. worker/clean.py applies rules (a) to (e) in id order; each rejected record is counted once, under the first rule it fails, so input = (a)+(b)+(c)+(d)+duplicates+kept holds (checked by an assertion). *Results.* PENDING: counts table (input / a / b / c / d / duplicates / kept), SHA-256 of train.jsonl, labelling wall-clock time. Verified so far: the cleaning logic and its determinism on synthetic shards (unit tests; not real data). *Limitations.* Teacher errors are kept (the cleaner only checks form, not truth); JSON mode makes (a) rarer than free text would. *Improvements.* Self-consistency or a second teacher pass for disagreements.

3. Task 2: Knowledge distillation and training

Implementation. make train reads only training/config.yaml (seed, learning rate 1e-4, batch 8, epochs, max length 128, library versions). Each example is Command: <text>\nJSON: <teacher JSON> plus EOS; the loss covers only the JSON tokens. The teacher JSON is re-serialised on one line (same content) so that generation can stop at EOS. Version v1 uses seed 42 / 3 epochs, v2 seed 7 / 4 epochs. The model files are stored in models/vN/ of SeaweedFS and as a deterministic model-vN.tar.gz Release asset whose SHA-256 is in models/vN/model.sha256. *Results.* PENDING: epoch losses (training/loss.csv); validation table below.

Model (100 validation commands)	json_validity	intent	date	time	place	full_frame
Teacher phi3.5	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING
Untrained Pythia-70M	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING
Student v1	PENDING	PENDING	PENDING	PENDING	PENDING	PENDING

Discussion. PENDING (what changed after distillation). **Sequence-level vs logit-based distillation.** Sequence-level distillation trains the student with ordinary cross-entropy on the text the teacher generated (Kim and Rush, 2016): the teacher is only a label generator. Logit-based distillation trains the student to match the teacher's per-token probability distribution (e.g. KL divergence), which carries more information about alternatives. It is not possible here because Ollama returns only text, and phi3.5 and Pythia use different tokenizers, so their token distributions cannot be aligned without extra machinery. A limitation of my approach: the student only sees one hard answer per command and inherits the teacher's mistakes; with 1,500 examples and 70M parameters it cannot be expected to generalise to unusual phrasing.

4. Task 3: Containerised inference service

service/app.py (FastAPI, port 8000): POST /parse returns the four-key JSON and headers X-Model-Version, X-Served-By (Pod name); /healthz is 200 while the process lives; /readyz is 503 until the model is loaded in a background thread. At start-up the model is downloaded from MODEL_URI on S3_ENDPOINT; credentials come from the Secret s3-credentials. Every output is checked against data/schema.json (service/validator.py); failures return HTTP 500 {"error":"invalid_model_output"}, and FAULT_INJECT=1 replaces the output with not json. Dependencies are pinned (CPU-only torch wheel); the image has no weights. *Results.* Verified: validator unit tests (5 pass). PENDING: API tests with FastAPI (4 tests, skipped in the authoring sandbox), image size, 50-request run at 1 CPU / 1 GiB, peak memory, OOM check. *Limitations.* One request at a time per Pod (a lock around generation); dependencies are pinned directly, a full pip freeze lock is a possible hardening.

5. Task 4: Kubernetes deployment and reliability

Deployment (2 replicas, requests 500m / 768Mi, limits 1 CPU / 1Gi), Service, Traefik Ingress for /parse, liveness /healthz, readiness /readyz, HPA 1 to 2 at 70% CPU. Rolling update uses maxSurge 1, maxUnavailable 0 so a Pod leaves only when a ready one replaces it. Because the readiness probe fails while loading, the Service has no endpoint for a Pod that is still downloading its model. *Results.* PENDING: scale-up/down evidence, rolling update failure rate (target ≤ 0.5%), rollback header v1, recovery (≤ 2% failures), three cold-start times and median, per-replica share (≥ 30% each). **Limitation (stated as required).** The cluster is one k3d node on one laptop: it demonstrates Kubernetes mechanics (probes, rescheduling, rolling updates) but not fault tolerance across machines, and the HPA scales Pods that compete for the same CPU, so it cannot add real capacity.

6. Task 5: Subnetting and NACL (executed and verified)

Allocation, largest first from 10.20.0.0, smallest prefix that fits (usable = 2^(32-p) - 2):

Subnet	Hosts	Prefix	Network	Usable range	Broadcast	Usable
app	1,000	/22	10.20.0.0	10.20.0.1 to 10.20.3.254	10.20.3.255	1,022
public	250	/24	10.20.4.0	10.20.4.1 to 10.20.4.254	10.20.4.255	254
data	100	/25	10.20.5.0	10.20.5.1 to 10.20.5.126	10.20.5.127	126
management	20	/27	10.20.5.128	10.20.5.129 to 10.20.5.158	10.20.5.159	30

NACL (public subnet), inbound: 100 deny 198.51.100.0/24; 110 allow TCP 80; 120 allow TCP 443; 130 allow TCP 22 from 10.20.5.128/27; 140 allow TCP 1024-65535 from 10.20.0.0/22 (replies from app). Outbound: 100 allow TCP 8000 to 10.20.0.0/22; 110 allow TCP 1024-65535 to 0.0.0.0/0 (replies to clients). *Measured:* nacl_sim.py --check returned 6/6 correct on the sample packets (the hidden 20 packets were not available). A NACL is stateless: it keeps no connection table, so a reply is a new packet judged by its own rules, hence the ephemeral-port rules. Rules are evaluated from the lowest number and the first match wins, so the deny for 198.51.100.0/24 must be numbered below the allows for ports 80 and 443; otherwise the broad allow would match first.

7. Task 6: Testing and CI/CD

make up builds and deploys from a fresh clone (one command; make down deletes the cluster). The workflow .github/workflows/ci.yml runs gitleaks, ruff, pytest (15 tests), model evaluation using the Release asset (score.py on validation.jsonl), then the gate score.py --

min-validity 95. *Verified here:* 11 unit tests pass, 4 skipped (no FastAPI in the authoring sandbox). *PENDING:* first CI run, the failed-gate run with the untrained model (link: PENDING), gitleaks result.

8. Task 7: Performance, cost and review

PENDING (all values must come from results/loadtest/*, kubectl top pods and scripts/cost.py): p50, p95, requests/s, error rate, CPU and memory per Pod, cold start, cost per 1,000 requests = (hourly cost of all replicas / requests per hour) x 1000 with every step shown (rate card: 3.50 INR/vCPU-hour, 0.45 INR/GiB-hour), SLO (target, window, error budget in requests, budget used).

Component	Evidence (measured)	Limitation	Improvement
Data and storage	PENDING		
Model quality	PENDING		
Latency and throughput	PENDING		
Resource utilisation	PENDING		
Reliability and recovery	PENDING		
Security and cost	PENDING		

9. Conclusion and references

PENDING conclusion. References: Kim and Rush, *Sequence-Level Knowledge Distillation*, EMNLP 2016; Bastianelli et al., *SLURP*, EMNLP 2020 (CC BY 4.0); Biderman et al., Pythia (EleutherAI/pythia-70m, Apache 2.0); k3d, SeaweedFS, Kubernetes, Locust documentation.